UNIVERSITÀ DI BOLOGNA



School of Engineering

Master Degree in Automation Engineering

Optimal Control and Reinforcement Learning

Course Project 2 - Flexible Robotic Link

Professor: **Giuseppe Notarstefano**

Students:

Francesco Cenni - 0001194000

Giulio Mazzaroppi - 0001189361

Giacomo Pezzolati - 0001190071

Academic year 2025/2026

Contents

1 Task 0: Problem Setup and Discretization **5**
- 1.1 Discretization Methods . . . 5
- 1.2 Analytic Jacobian Computation . . . 6

2 Task 1: Equilibria and Terminal Cost **7**
- 2.1 Equilibrium Manifold . . . 7
- 2.2 Cost Function . . . 8
- 2.3 Terminal Cost via DARE . . . 8
- 2.4 Step Size Selection (Armijo Rule) . . . 8

3 Task 2: Trajectory Generation (Newton's Method) **10**
- 3.1 Reference Generation . . . 10
  - 3.1.1 Task Definitions . . . 11
- 3.2 Performance Analysis: Stepwise vs. Polynomial . . . 11
- 3.3 Optimization Challenges . . . 13
- 3.4 Proposed Solution: Dynamic Horizon Extension . . . 15
  - 3.4.1 Theoretical Justification . . . 15
  - 3.4.2 Algorithm Implementation . . . 16

4 Task 3: Trajectory Tracking via LQR **18**
- 4.1 Linear Time-Varying (LTV) LQR . . . 18
- 4.2 Implementation Details . . . 18
- 4.3 Simulation Results . . . 19

5 Task 4: Trajectory Tracking via MPC **27**
- 5.1 MPC Formulation . . . 27
- 5.2 Key Features . . . 27
- 5.3 Simulation Results . . . 27

6 Task 5: Animation **34**
- 6.1 Visualization . . . 34

7 Conclusion **35**

8 Appendix **36**
- 8.1 Models matrices and parameters . . . 36
- 8.2 Analytical Derivation of the Jacobian . . . 36
  - 8.2.1 State-Space Formulation . . . 36
  - 8.2.2 Block A_{22}: Partial Derivative w.r.t. Velocities . . . 37
  - 8.2.3 Block A_{21}: Partial Derivative w.r.t. Positions . . . 37
  - 8.2.4 Jacobian Matrix B . . . 37
  - 8.2.5 Explicit Derivative Matrices . . . 37
- 8.3 Derivation of Discrete-Time Jacobians . . . 38
  - 8.3.1 Forward Euler Discretization . . . 38
  - 8.3.2 Runge-Kutta 4 (RK4) Discretization . . . 38
- 8.4 Newton Method with Dynamic Horizon Extension algorithm . . . 39

List of Figures

1 Double Pendulum System with flexible link. 5
2 3D Equilibrium manifold. 7
3 Equilibrium manifold as a function of θ_1. 7
4 Armijo line search behavior A. 9
5 Armijo line search behavior B. 9
6 Comparison of reference generation strategies. 10
7 Visualization of the defined control tasks. 11
8 Newton's method applied to Swing Up using Stepwise reference. Note the control discontinuity. 12
9 Optimization results using 5th-order polynomial references. 13
10 Illustration of the "Local Minimum Trap" where the solver converges to a suboptimal swing. 15
11 Evolution of the trajectory during the Dynamic Horizon Extension process. . . . 17
12 Trajectory tracking on a downward maneuver 19
13 Error analysis during LQR tracking of downward maneuver 20
14 Trajectory tracking on a swing-up maneuver 21
15 Error analysis during LQR tracking 22
16 Trajectory tracking on a hold maneuver 23
17 Error analysis during LQR tracking 24
18 Trajectory tracking on a elbow up maneuver 25
19 Error analysis during LQR tracking 26
20 Trajectory tracking on a dive maneuver 28
21 Error analysis during MPC tracking of dive maneuver 29
22 Trajectory tracking on a swing up maneuver 30
23 Error analysis during MPC tracking of swing up maneuver 31
24 Trajectory tracking on a elbow up maneuver 32
25 Error analysis during MPC tracking of a elbow maneuver 33

Abstract

This project addresses the optimal control and trajectory tracking of a flexible robotic link, modeled as an under-actuated planar double pendulum with nonlinear joint stiffness.

The goal of this work is to develop a comprehensive framework for trajectory generation and tracking control, considering the system's inherent nonlinearities and actuation constraints.

The study adopts a flexible discretization framework where the integration scheme is selected based on the specific requirements of the maneuver and its required fidelity. The analytic Jacobians are derived directly from the chosen discrete-time mapping, ensuring that the gradients remain consistent with the state-transition dynamics.

We analyze the system's equilibrium manifold to identify stable and unstable configurations.

Trajectory generation is performed using a Newton-based approach. Moreover, a novel Dynamic Horizon Extension strategy is proposed to enhance the convergence of trajectory optimization algorithms in the presence of complex dynamics.

Finally, we implement and compare two tracking controllers: a Linear Time-Varying Linear Quadratic Regulator (LTV-LQR) for unconstrained stabilization, and a Model Predictive Control (MPC) scheme to handle actuator saturation.

Simulation results demonstrate the efficacy of the proposed extension strategy in generating feasible trajectories and highlight the superior performance of MPC in tracking under input constraints.

Introduction

This report presents the design and implementation of optimal control strategies for a flexible robotic link. The system is modeled as a planar double pendulum where only the first joint is actuated, and a nonlinear elastic behavior is present at the end of the second link.

This structure introduces significant control challenges, specifically:

- **Under-actuation:** The system has two degrees of freedom (DoF) but only one control input (torque at the base);
- **Nonlinear Elasticity:** The stiffness between the links depends nonlinearly on the relative angle θ_2, complicating the dynamics compared to standard linear springs;
- **Non-controllability:** The system is not fully feedback linearizable, and maintaining equilibrium requires precise balancing of gravitational and elastic forces;
- **Low Friction:** The provided parameters (Set 2) involve relatively low friction coefficients, making the system prone to oscillations and sensitive to disturbances.

1 Task 0: Problem Setup and Discretization

The double pendulum model used in this project is reported in Figure 1.



Figure 1: Double Pendulum System with flexible link.

The configuration vector is defined as: $q = \begin{bmatrix}\theta_1 & \theta_2\end{bmatrix}^\top \in \mathbb{R}^n$. With $n = 2$ degrees of freedom. The continuous-time dynamics of the double pendulum are governed by the equation:

$$M(q)\ddot{q} + C(q,\dot{q})\dot{q} + F\dot{q} + G(q) = \begin{bmatrix}\tau \\ 0\end{bmatrix} \tag{1}$$

Where[1]:

- $M(q) = M(q)^\top \succeq 0 \in \mathbb{R}^{n\times n}$ is the inertia matrix;
- $C(q,\dot{q}) = C(q,\dot{q})^\top \succeq 0 \in \mathbb{R}^{n\times n}$ represents Coriolis and centrifugal forces;
- $F = F^\top \succeq 0 \in \mathbb{R}^{n\times n}$ is the viscous friction matrix;
- $G(q)$ is the gravity vector;
- $\tau \in \mathbb{R}^{n_i}$ is the control torque applied at the first joint[2] with $n_i = 1$.

The adopted parameters for this system are summarized in Table 1.

The state space vector is defined to include both positions and velocities: $x = \begin{bmatrix}\theta_1 & \theta_2 & \dot{\theta}_1 & \dot{\theta}_2\end{bmatrix}^\top \in \mathbb{R}^{n_s}$. With $n_s = 2n = 4$ states. Thus, the continuous time state-space nonlinear dynamics can be expressed as:

$$\dot{x} = f(x,u) = \begin{bmatrix}\dot{q} & \ddot{q}\end{bmatrix}^\top = \begin{bmatrix}\dot{\theta}_1 & \dot{\theta}_2 & \ddot{\theta}_1 & \ddot{\theta}_2\end{bmatrix}^\top. \tag{2}$$

Where:

- $u = \tau \in \mathbb{R}$ control input;
- $f : \mathbb{R}^{n_s} \times \mathbb{R} \to \mathbb{R}^{n_s}$ is a nonlinear function representing the system dynamics.

1.1 Discretization Methods

Two discretization schemes has been implemented in `solver.py`[3].

[1] The detailed expressions for $M(q)$, $C(q,\dot{q})$, F, and $G(q)$ can be found in the Appendix 8.1.
[2] In this tractation are not considered non-conservative forces or contact forces.
[3] Details in Appendix 8.3

1. **Forward Euler (FE):**

$$x_{k+1} = x_k + \Delta t f(x_k, u_k) \tag{3}$$

While simple, this method is numerically unstable for stiff systems or large time steps;

2. **Runge-Kutta 4th Order (RK4):**

$$x_{k+1} = x_k + \frac{\Delta t}{6}(k_1 + 2k_2 + 2k_3 + k_4) \tag{4}$$

This method maps all poles of the continuous-time system to within the unit circle in the discrete-time domain, ensuring stability. However, its computation time is higher that FE.

Where:

- Δt is the discretization time step;
- x_k, u_k are the state and control input at time step k;
- k_1, k_2, k_3, k_4 are the RK4 intermediate slopes.

1.2 Analytic Jacobian Computation

For the Newton-based optimal control algorithm, accurate gradients of the dynamics are required. Relying on finite differences can introduce numerical noise, while symbolic differentiation (via `SymPy`) proved too slow for iterative solving.

Therefore, we implemented an **analytic Jacobian** computation in `model.py`.

- $A = \frac{\partial f(x,u)}{\partial x} = \nabla_1 f(x,u)^\top \in \mathbb{R}^{n_s \times n_s}$: State Jacobian;
- $B = \frac{\partial f(x,u)}{\partial u} = \nabla_2 f(x,u)^\top \in \mathbb{R}^{n_s \times n_i}$: Input Jacobian;

The Jacobians were derived manually and implemented using optimized NumPy operations (and Numba JIT compilation for speed). This approach ensures that the linearization required for the Backward Pass in Newton's method is both precise and computationally efficient. The detailed derivation is reported in Appendix 8.2.

2 Task 1: Equilibria and Terminal Cost

2.1 Equilibrium Manifold

The system's equilibria are defined by states where $\dot{q} = \ddot{q} = 0$. This implies the gravity and stiffness torques must be balanced by the input τ. We analyzed the equilibrium manifold by fixing one generalized coordinate (θ_1 or θ_2) and solving for the other.

$$\cancel{M(\theta_1,\theta_2)\begin{pmatrix}\ddot{\theta}_1\\ \ddot{\theta}_2\end{pmatrix}} + \cancel{C(\theta_1,\theta_2,\dot{\theta}_1,\dot{\theta}_2)\begin{pmatrix}\dot{\theta}_1\\ \dot{\theta}_2\end{pmatrix}} + \cancel{F\begin{pmatrix}\dot{\theta}_1\\ \dot{\theta}_2\end{pmatrix}} + G(\theta_1,\theta_2) = \begin{pmatrix}0\\ \tau\end{pmatrix} \tag{5}$$

This reduces to:

$$\underbrace{\begin{pmatrix} gl_{c1}m_1\sin(\theta_1) + gm_2(l_1\sin(\theta_1) + l_{c2}\sin(\theta_1+\theta_2)) \\ gm_2l_{c2}\sin(\theta_1+\theta_2) + 3k\sin(\theta_2)\cos^2(\theta_2)\end{pmatrix}}_{G(\theta_1,\theta_2)} = \underbrace{\begin{pmatrix}0\\ \tau\end{pmatrix}}_{\boldsymbol{\tau}} \tag{6}$$

In this framework, we have three variables: θ_1, θ_2, and τ. By fixing one of the three variables, we can solve for the other two. It looks natural, since we are interested in obtaining specific configurations, and since the first joint is actuated, to fix θ_1 and solve for θ_2 and τ.

Using this approach, we generate an equilibrium manifold that is a function of the first joint angle θ_1. It is worth noting that two distinct configurations arise for each θ_1 value, corresponding to different θ_2 angles (e.g., "Elbow-Up" and "Elbow-Down" positions). They will respectively be stable and unstable equilibria.

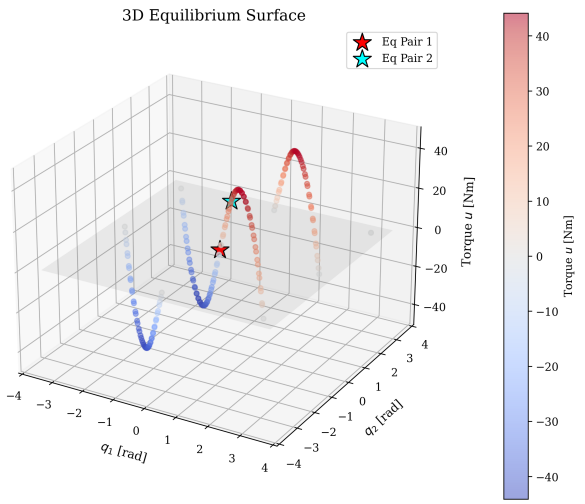


Figure 2: 3D Equilibrium manifold.

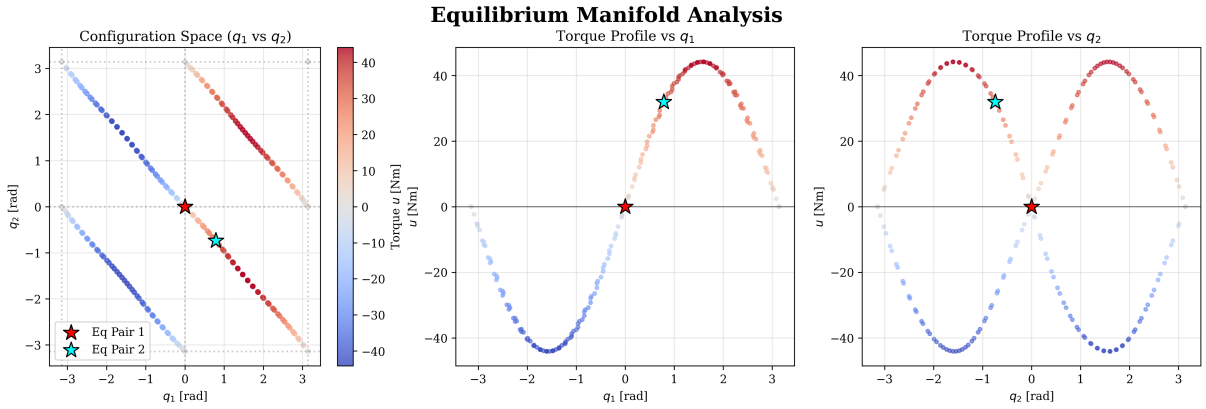


Figure 3: Equilibrium manifold as a function of θ_1.

The first thing we can see is that the manifold has a sinusoidal shape and a periodic behavior, which is expected given the rotational nature of the joints and the influence of gravity. From the plot, we can also observe some important characteristics:

- **Branches** We can say that the manifold is **constituted by branches**, hence there will be starting and ending equilibria that are on the same branch and others that will be in two different ones.
  This will have an impact on the trajectory generation, as moving between different branches may **require more complex maneuvers**.

- **Uncontrolled second arm** Since the second joint is unactuated, the torque τ is only applied at the first joint.
  This means that the system's ability to reach certain **configurations** in a given **time** is **limited by the dynamics of the second link** and the coupling between the two joints. This coupling will prove useful in achieving a stabilization of the second link by appropriately controlling the first one. We will see this is **not always possible**, and this causes problems in trajectory generation when trying to reach certain target equilibria and mantain them for some time.

2.2 Cost Function

For the optimal control problem, we defined the quadratic cost function:

$$J = \sum_{t=0}^{T-1} \left(\frac{1}{2} \Delta x_t^\top Q \Delta x_t + \frac{1}{2} \Delta u_t^\top R \Delta u_t \right) + \frac{1}{2} \Delta x_T^\top Q_T \Delta x_T \tag{7}$$

where $\Delta x = x - x_{ref}$ and $\Delta u = u - u_{ref}$. We selected $Q = \text{diag}(1, 1, 1, 1)$ and $R = 10^{-3} I$ to penalize state deviation while allowing for aggressive control actions.

2.3 Terminal Cost via DARE

A critical aspect of the finite-horizon optimization is the terminal cost Q_T. Simply setting $Q_T = Q$ often fails to stabilize the system at the end of the horizon, especially for short durations. In fact, without an appropriate terminal cost, the solver may produce trajectories that approach but do not settle at the desired equilibrium, leading to oscillations or drift post-optimization. It is as if the optimizer result was **"myopic"**, focusing only on minimizing cost up to time T without considering the long-term behavior beyond this point. To address this, we computed Q_T by solving the **Discrete Algebraic Riccati Equation** (DARE) based on the linearization at the final target equilibrium (x_{eq}, u_{eq}).

$$P_\infty = A^\top P_\infty A - (A^\top P_\infty B)(R + B^\top P_\infty B)^{-1}(B^\top P_\infty A) + Q \tag{8}$$

Setting $Q_T = P_\infty$ provides an approximation of the infinite-horizon cost-to-go. This essentially informs the solver that the trajectory continues optimally after time T, significantly improving convergence and stability for tasks like the "Swing Up".

2.4 Step Size Selection (Armijo Rule)

To ensure global convergence of the Newton method, we implemented a **line search using the Armijo rule**. The step size α is reduced ($\alpha \leftarrow \beta\alpha$) until the actual cost reduction is a sufficient fraction of the predicted reduction (based on the gradient). This prevents the solver from taking overly aggressive steps that could destabilize the trajectory and **ensures steady progress towards the optimum**. Additionally, the implementation includes a gradient check that validates the analytic descent direction by comparing it to a numeric slope $\frac{J(u+\epsilon\Delta u)-J(u)}{\epsilon}$ computed via finite difference. If the Armijo search fails to satisfy the sufficient decrease condition within the maximum allowed iterations, the **step size is set to zero** to prevent the update and safeguard the solver from adopting an unstable or divergent trajectory. Here there are some plots of the Armijo line search behavior during the trajectory optimization.

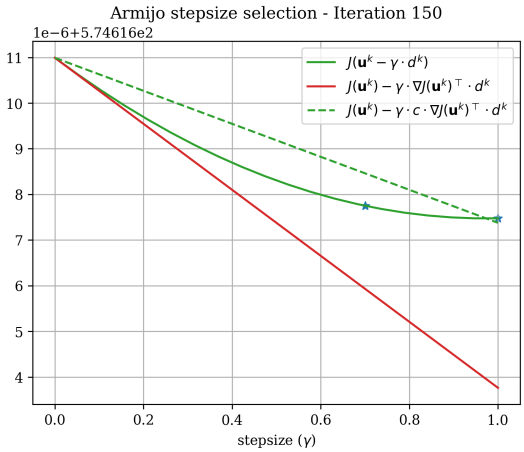


Figure 4: Armijo line search behavior A.

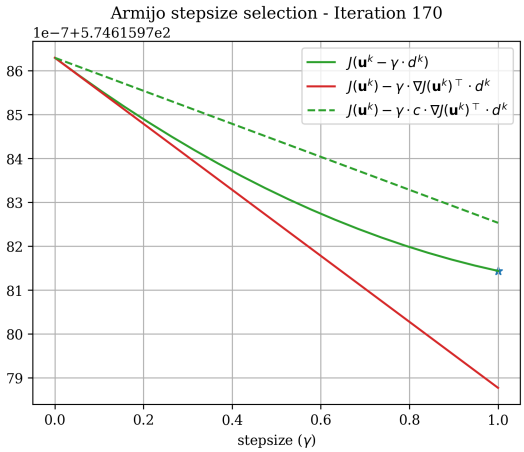


Figure 5: Armijo line search behavior B.

3 Task 2: Trajectory Generation (Newton's Method)

We employed a Newton-type method for optimal control, specifically a Differential Dynamic Programming (DDP) approach, to generate feasible trajectories transitioning between system equilibria. This method iteratively optimizes the control sequence by utilizing quadratic approximations of the cost function and linear approximations of the system dynamics.

3.1 Reference Generation

Given the non-convex nature of the optimization landscape, the solver requires a reasonable reference trajectory to guide convergence, even if that reference is dynamically infeasible. We analyzed two distinct generation methods:

1. **Stepwise:** A naive approach involving an instantaneous switch from the initial state x_{start} to the goal state x_{goal}. This introduces a large discontinuity in the error term, often leading to poor convergence or aggressive control spikes.
2. **Polynomial Interpolation (Poly 5):** We generated smooth references for joint positions and velocities using 5th-order polynomials. This method ensures kinematic consistency by respecting boundary conditions (position, velocity), thereby significantly aiding the solver in finding a feasible solution.



(a) Stepwise Reference



(b) Poly 5 Reference

Figure 6: Comparison of reference generation strategies.

3.1.1 Task Definitions

We defined a set of standard maneuvers to evaluate the generation algorithms:



(a) Swing Up (b) Swing Down (c) Complex Maneuver

Figure 7: Visualization of the defined control tasks.

3.2 Performance Analysis: Stepwise vs. Polynomial

The choice of reference trajectory significantly impacts the solver's performance. When applying Newton's method to the **Stepwise** reference, we observed significant limitations. As seen in Figure 8, the instantaneous jump in the reference creates a large error gradient at the transition point. The solver attempts to compensate with an infeasible spike in control torque u, often pushing the system into unstable regions.



Figure 8: Newton's method applied to Swing Up using Stepwise reference. Note the control discontinuity.

Conversely, the **Polynomial (Poly 5)** reference provides a physically plausible kinematic guide. However, physical constraints inherent to the under-actuated flexible link still present challenges:

- **Energy Transfer (Elbow Up):** During swing-up motions, kinetic energy must be efficiently transferred to potential energy. The polynomial reference guides this transfer smoothly.
- **Energy Dissipation (Elbow Down):** In downward motions, the passive second link cannot be directly braked. Since the system friction is low, the controller cannot rely on passive dissipation; it must actively pump energy out of the system via coupling, which is difficult if the reference demands a stop faster than the system's natural dynamics allow.



(a) Poly 5 applied to Swing Up



(b) Poly 5 applied to Swing Down

Figure 9: Optimization results using 5th-order polynomial references.

3.3 Optimization Challenges

Applying the standard Newton method directly to complex, long-horizon maneuvers revealed several critical numerical and theoretical issues:

- **Non-Convex Optimization Landscape:** For complex maneuvers, the trajectory manifold is highly non-linear. The optimization landscape is riddled with local minima. If the initial guess falls within the basin of attraction of a local minimum, representing a partial swing or a stalled state, the solver is unable to escape to the global optimum.
- **Long Time Horizons and Sensitivity:** Extending the time horizon T exponentially increases the complexity of the solution space. A short horizon simplifies the optimization but fails to stabilize the system exactly at the equilibrium. Conversely, a long horizon increases the condition number of the Hessian matrix. Small changes in initial inputs propagate through the sensitivity matrix $\prod_{k=0}^{T-1} A_k$, causing massive deviations in terminal states and leading to numerical divergence.
- **Controllability and Terminal Cost:** To implement a stabilizing terminal cost Q_T, we solve the Discrete Algebraic Riccati Equation (DARE):

$$P = A^\top PA - (A^\top PB)(R + B^\top PB)^{-1}(B^\top PA) + Q \tag{9}$$

  This requires the system linearization (A, B) at the target equilibrium to be stabilizable.

In under-actuated configurations, kinematic singularities can cause rank deficiencies in the controllability matrix, making the calculation of Q_T impossible.

- **Instability to Dwell Time:** We observed numerical instability when requiring the system to maintain an unstable equilibrium for a final time interval (dwell time). Consider the analogy of balancing a ball on a sharp peak: reaching the peak at time T requires precision, but remaining there requires perfect force cancellation over time. Any numerical error ϵ at $t < T$ is amplified by the unstable eigenvalues of the system evolution matrix (A^k). This creates a "narrow ridge" in the cost landscape; slight deviations result in massive penalties, pushing variables into numerical overflow.

- **Linearization Validity:** The optimization relies on linearizing the non-linear dynamics around the current trajectory iteration. If the update step is too large, or if the trajectory deviates significantly into highly non-linear regions, the Taylor approximation breaks down, leading to valid descent directions becoming actual ascent directions in the non-linear cost landscape.



Figure 10: Illustration of the "Local Minimum Trap" where the solver converges to a suboptimal swing.

3.4 Proposed Solution: Dynamic Horizon Extension

To address the ill-conditioning and local minima problems described above, we implemented a **Dynamic Horizon Extension** strategy.

Instead of attempting to solve the full problem T_{final} immediately, we solve a sequence of sub-problems with increasing horizon lengths. This strategy conditions the optimization by ensuring the solver always operates within a valid local approximation.

3.4.1 Theoretical Justification

1. **Conditioning:** For a short horizon T, the sensitivity matrix norm remains bounded, resulting in a well-conditioned Hessian and a larger basin of attraction for Newton's method.

2. **Warm Starting:** The optimal solution for horizon T is used as the initial guess for

$T + \Delta T$. This "anchors" the new optimization in a feasible region of the state space, close to the physical constraints.

3. **Iterative Refinement:** As the horizon expands, the solver primarily refines the control inputs locally. This avoids the explosive divergence associated with global searches in ill-conditioned landscapes.

3.4.2 Algorithm Implementation

The procedure is defined as follows:

1. **Initialize:** Set a short initial horizon $T_{curr} \ll T_{final}$ where the DARE terminal cost effectively captures the remaining cost-to-go.

2. **Converge:** Execute the Newton solver until convergence criteria are met for the current T_{curr}.

3. **Extend:** Use the computed trajectory $x^*(t)$ as the initial guess. Extend the horizon by ΔT, padding the reference tail with the target equilibrium and evolving the initial guess via the system dynamics.

4. **Repeat:** Increment $T_{curr} \leftarrow T_{curr} + \Delta T$ and repeat until $T_{curr} \geq T_{final}$ and the terminal error satisfies $\|\mathbf{x}_T - \mathbf{x}_{target}\| < \epsilon$.



Figure 11: Evolution of the trajectory during the Dynamic Horizon Extension process.

4 Task 3: Trajectory Tracking via LQR

Once an optimal trajectory (x^*, u^*) is generated, we designed a **Linear Quadratic Regulator** (LQR) to stabilize the system around this trajectory in the presence of **disturbances**.
The main objective is to **minimize deviations** from the reference while ensuring **smooth control actions**, also ensuring stability at the end of the maneuver is **maintained or recovered**.

4.1 Linear Time-Varying (LTV) LQR

Since the reference trajectory is not a static point, we linearized the system dynamics along the trajectory, resulting in time-varying matrices A_t and B_t:

$$\delta x_{t+1} = A_t \delta x_t + B_t \delta u_t \tag{10}$$

The feedback law is computed as $u_t = u_t^* - K_t(x_t - x_t^*)$, where K_t is obtained by solving the Riccati equation backward in time. The input is therefore composed of the nominal **feedforward** term u_t^* and a **feedback** correction based on the state deviation.

4.2 Implementation Details

- **Perturbations:** To evaluate the controller's basin of **attraction**, we initialized the system with stochastic perturbations ($\epsilon \sim U[-0.05, 0.05]$) and then, for each run, we decided to apply the following initial disturbance:

  $$x_0 = x_0^* + \epsilon \cdot p_{\text{scale}} \cdot rr \tag{11}$$

  Where rr is the specific run, and p_{scale} is a scaling factor. By scaling these perturbations across multiple runs, we observed the LQR's ability to recover the nominal path even when the initial state deviated significantly from the optimal starting point.

- **Horizon Extension and Terminal Stability:** To ensure consistent behavior between the trajectory generation and tracking phases, the terminal cost matrix Q_T was defined as the **solution to the Discrete Algebraic Riccati Equation** (DARE) computed at the final equilibrium point.
  This choice effectively incorporates an infinite-horizon cost-to-go into the finite-horizon LQR framework, allowing the controller to **transition from a trajectory tracker into a steady-state regulator**.
  As shown in the extended simulations, the LQR continues to operate beyond the nominal time T to demonstrate the *asymptotic stability* of the closed-loop system.
  It is important to note that the validity of this terminal regulation relies on the **final trajectory state being a true equilibrium point**.
  This ensures that the linearized dynamics used for the DARE remain a faithful approximation of the nonlinear system, allowing the LQR to stabilize the robot within the neighborhood of the target pose even in the presence of residual perturbations.

- **Cost Function:** The weighting matrices Q, R, and Q_T were chosen to **balance state deviation penalties and control effort**, tuned through simulations to achieve desired performance.
  In this case, differently from the trajectory generation phase, we opted for a higher weight on the control effort (R matrix) to **avoid aggressive maneuvers** during tracking.
  Furthermore, given the flexible nature of the robot, high-gain tracking ($Q \gg R$) could **excite high-frequency oscillations** or lead to saturation in a real-world scenario.

4.3 Simulation Results

The results (see Figure 12, 14, 16) demonstrate that the LTV-LQR effectively rejects initial disturbances and tracks the complex optimal maneuver. It is possible to observe how the final stabilization of the second term is left onto the first arm, which oscillates to neglect the perturbations on the second link, and only later it will stabilize. However, there are cases in which the initial perturbation is too large for the LQR to recover the nominal trajectory, leading to divergence (see Figure 18).



Figure 12: Trajectory tracking on a downward maneuver

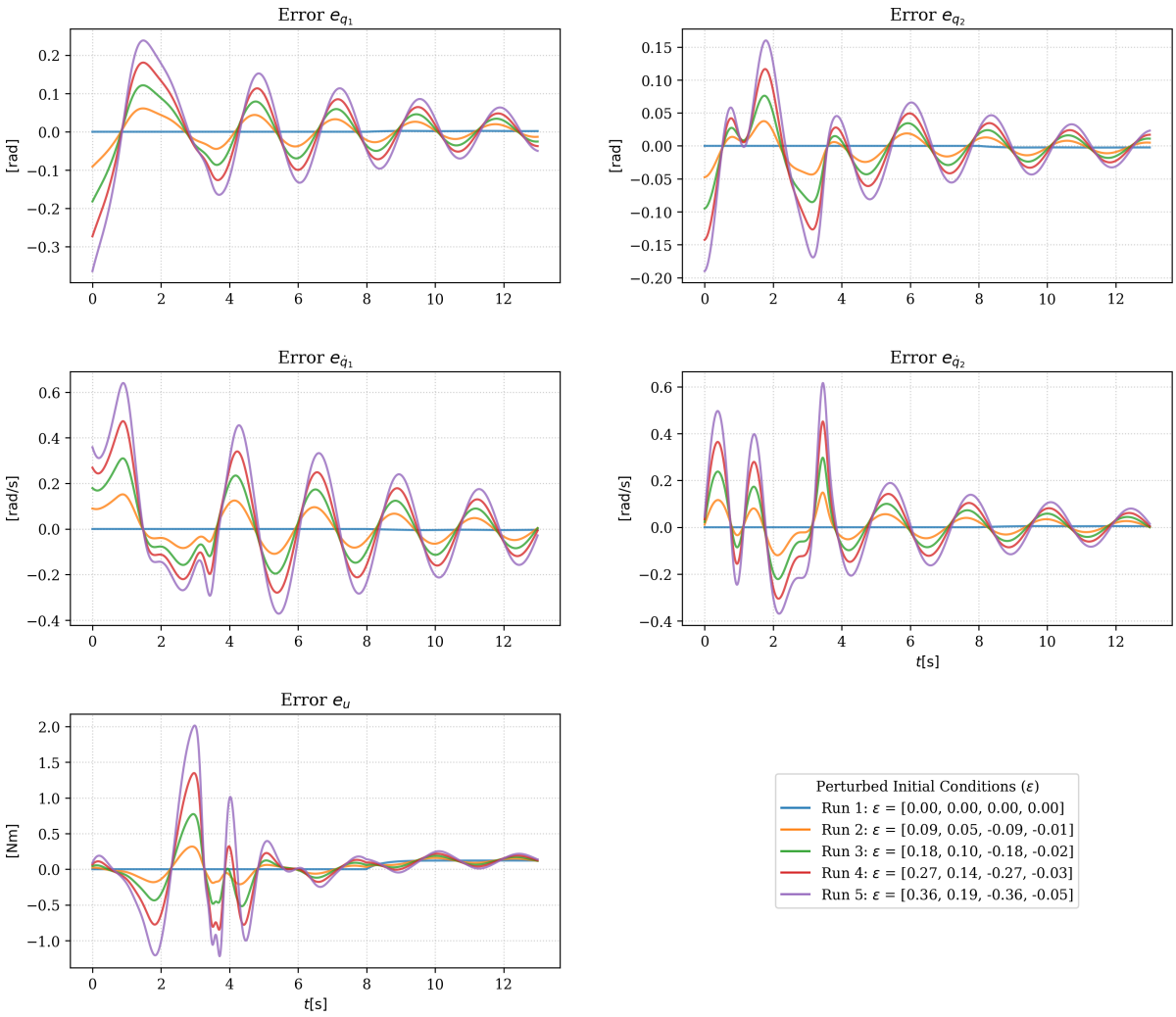


Figure 13: Error analysis during LQR tracking of downward maneuver

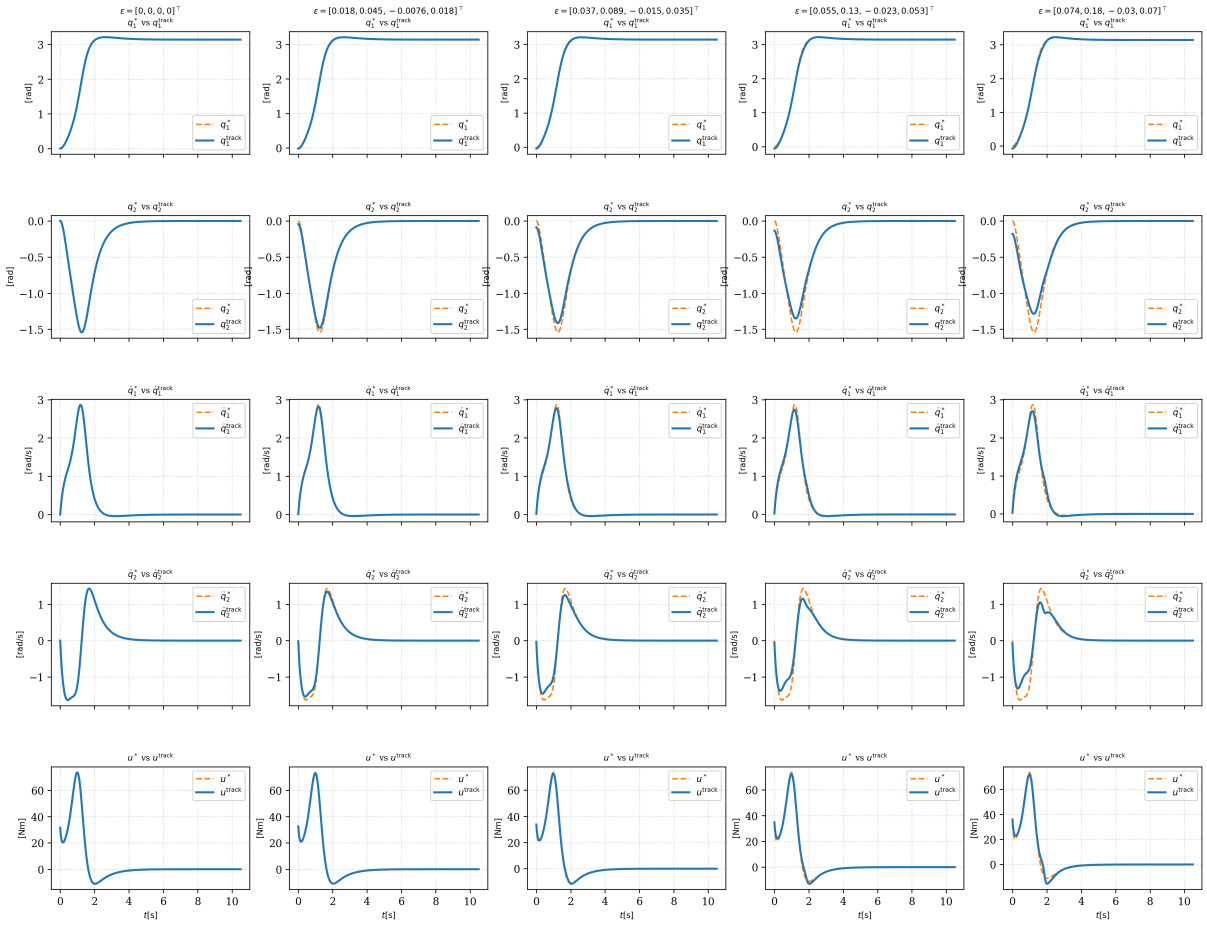


Figure 14: Trajectory tracking on a swing-up maneuver

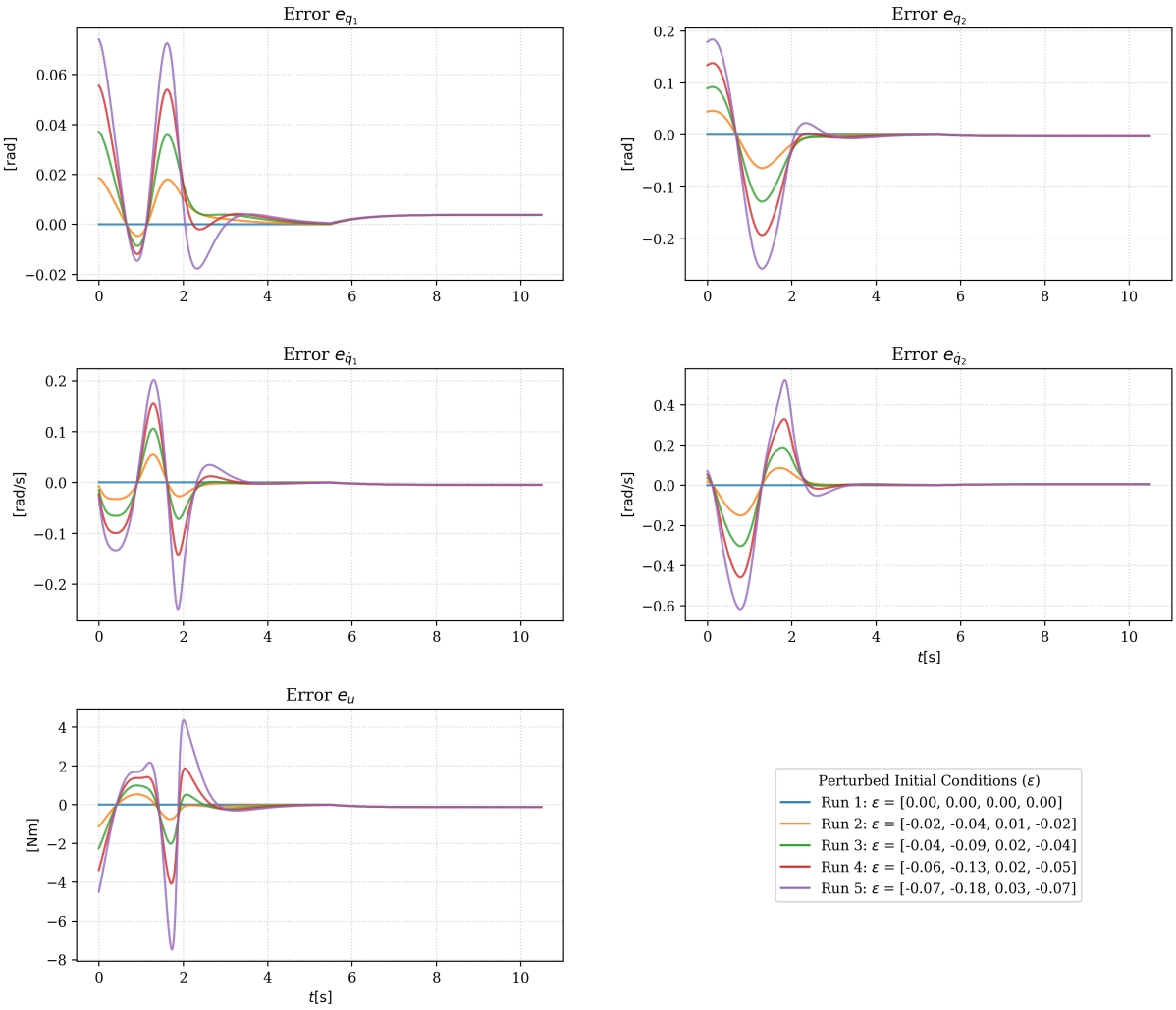


Figure 15: Error analysis during LQR tracking

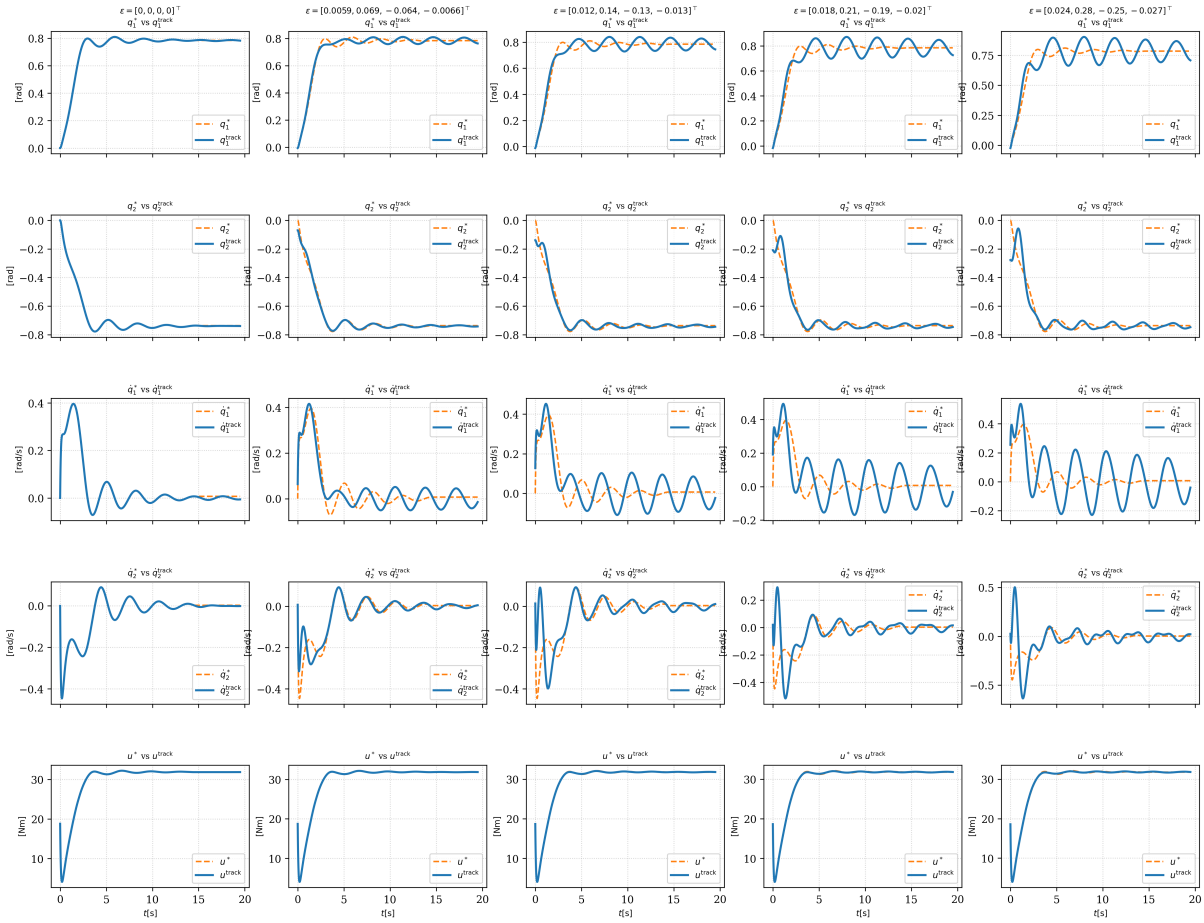


Figure 16: Trajectory tracking on a hold maneuver

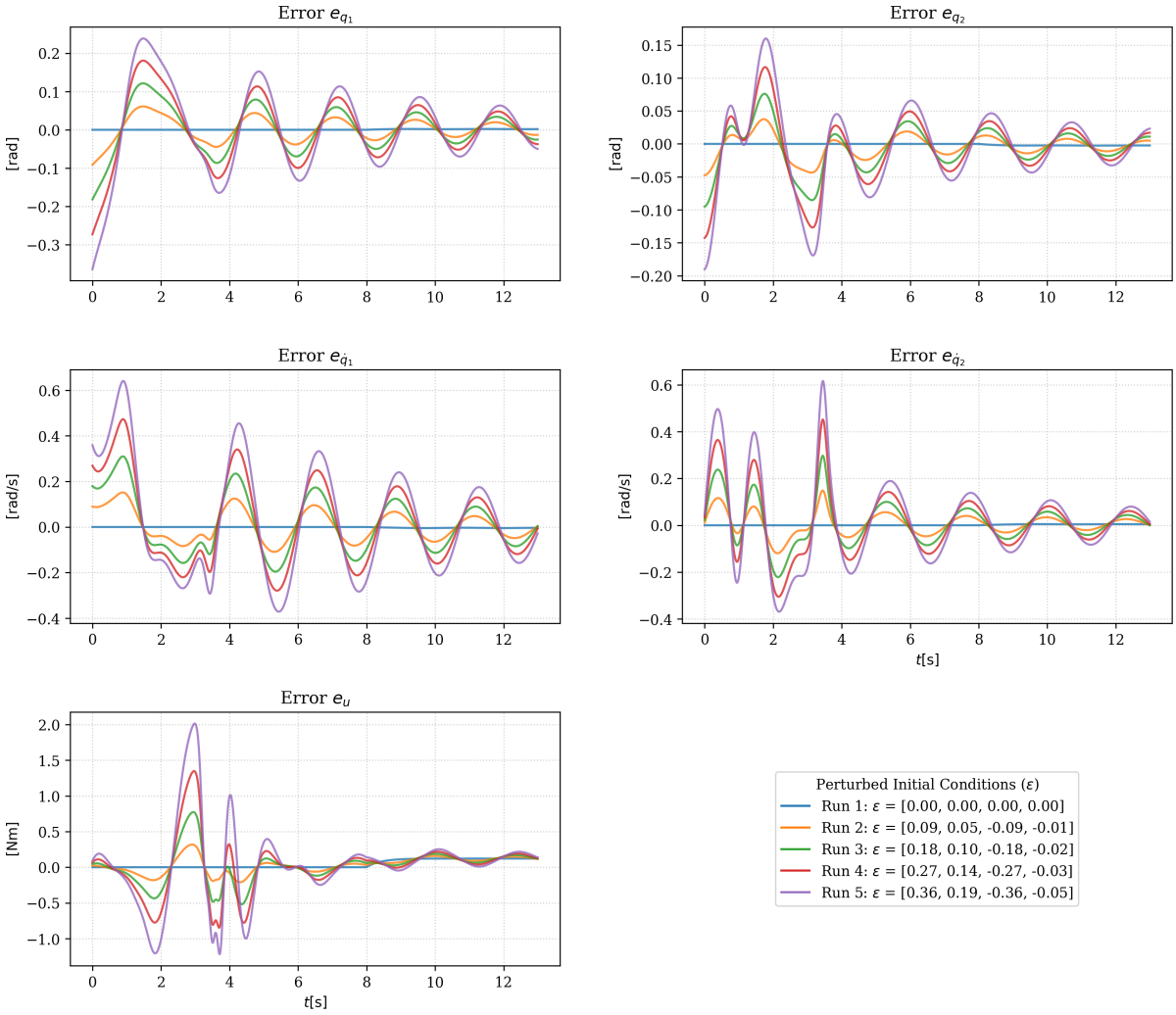


Figure 17: Error analysis during LQR tracking

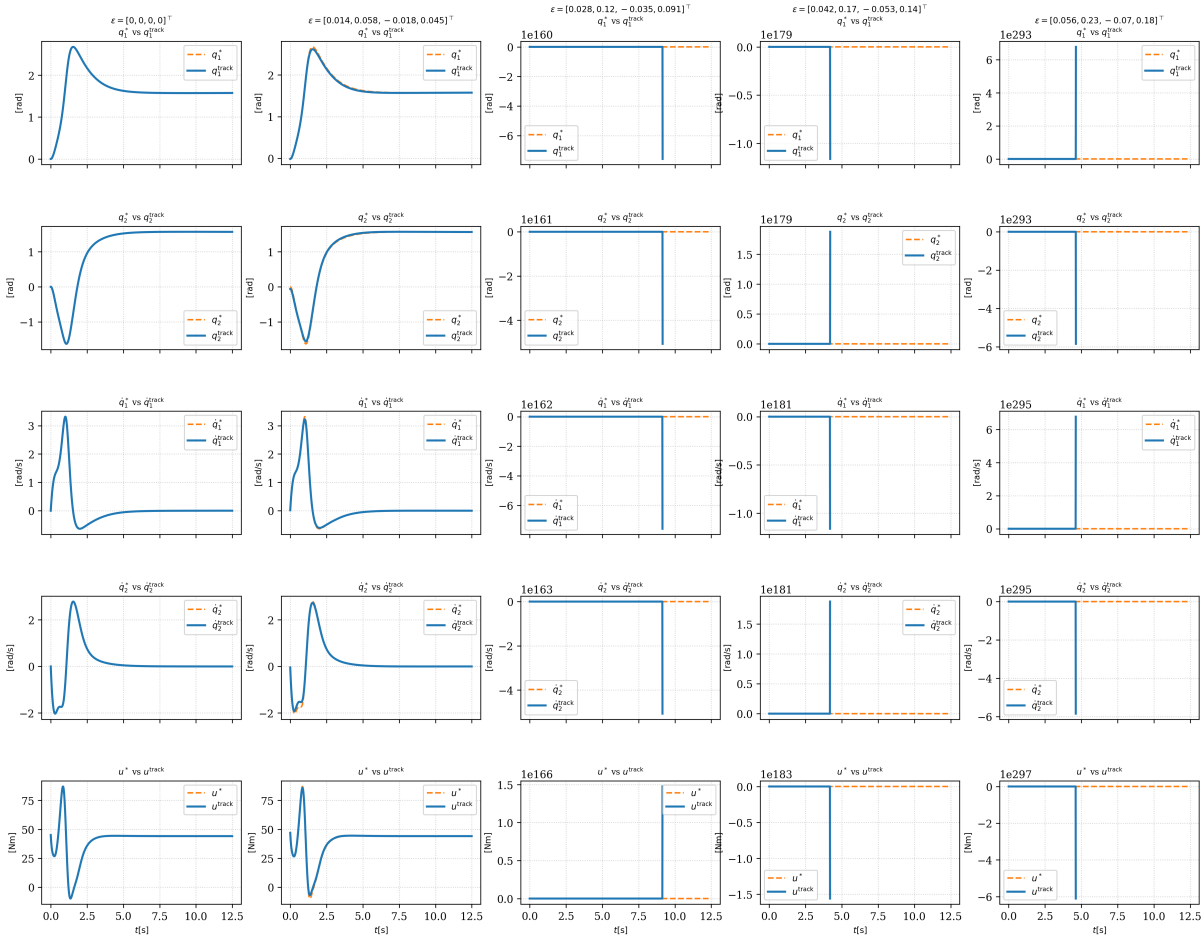


Figure 18: Trajectory tracking on a elbow up maneuver

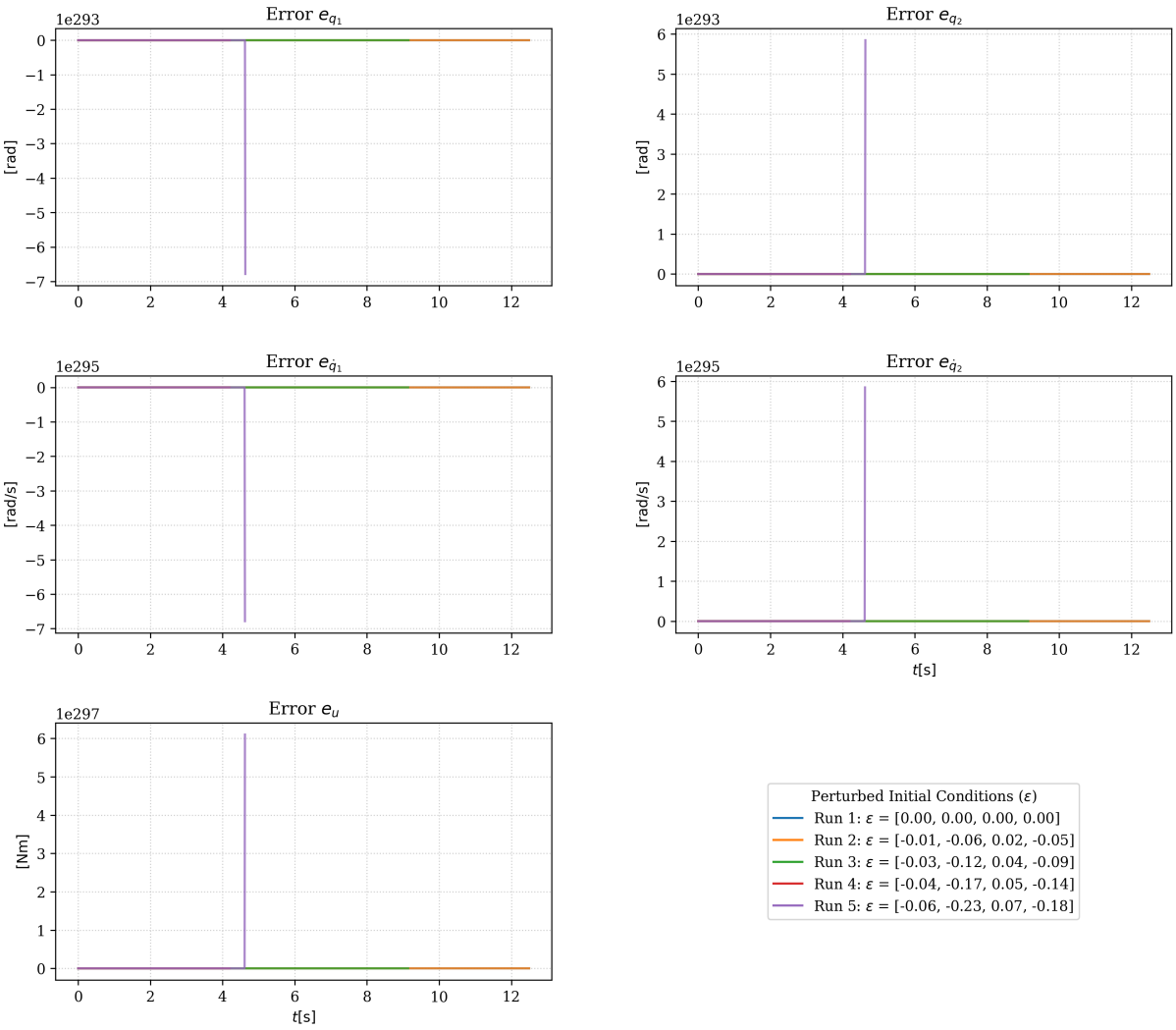


Figure 19: Error analysis during LQR tracking

5 Task 4: Trajectory Tracking via MPC

While LQR is effective for unconstrained tracking, it **cannot explicitly handle actuator saturation**. We implemented a linear **Model Predictive Control** (MPC) scheme to address this.

5.1 MPC Formulation

At each time step t, we solve a finite-horizon optimization problem:

$$\begin{aligned} \min_{\Delta u} \quad & \sum_{t=\tau}^{\tau+T_{pred}} \left(\|\Delta x_\tau\|_Q^2 + \|\Delta u_\tau\|_R^2 \right) + \|\Delta x_{T_{pred}}\|_{Q_T}^2 \\ \text{s.t.} \quad & \Delta x_{t+1} = A_t \Delta x_t + B_t \Delta u_t, \quad \forall t = \tau, ..., \tau + T_{pred} \\ & u_{min} \leq u_t^* + \Delta u_t \leq u_{max}, \quad \forall t = \tau, ..., \tau + T_{pred} \end{aligned}$$

where T_{pred} is the prediction horizon.

5.2 Key Features

- **Constraints:** We imposed hard constraints on the input torque ($u \in [u_{min}, u_{max}]$) to **respect physical motor limits**, which LQR ignores.
  In particular, we set $u_{max} = \max(|u^*|)$ and $u_{min} = -u_{max}$, so as **not to exceed the torque needed to complete the nominal trajectory** with no disturbances; managing to find a way through thanks to the prediction window and the optimization.
- **Receding Horizon:** Only the first control input is applied, and the optimization is repeated at the next step. Furthermore we opted for a prediction horizon T_{pred}
  that could serve as a **good compromise between performance and computational load**. In particular, we set $T_{pred} = \min(100, \frac{\text{len}(time)}{2})$.
- **Perturbations:** Similar to the LQR case, we introduced random initial state perturbations to evaluate robustness. We opted for bigger values, about 1.5 times those used in the LQR case.
- **Solver:** We utilized CasADi with IPOPT to solve the constrained QP/NLP at each step.

5.3 Simulation Results

The MPC controller showed superior performance in scenarios where large perturbations would cause an LQR controller to request infeasible torques.
Moreover, the ability to handle constraints makes MPC a versatile choice.
Down below some results are shown (see Figure 20, 22, 24). One can notice how, in some cases, the knowledge of the future helps in recovering the reference trajectory and final equilibrium, and other cases in
which **the window and/or the constraints do not allow the controller to return to the nominal path** (as in Figure 22). Above all, the controller **never exceeds the maximum torque defined**, showing the effectiveness of the constraints in the optimization problem.

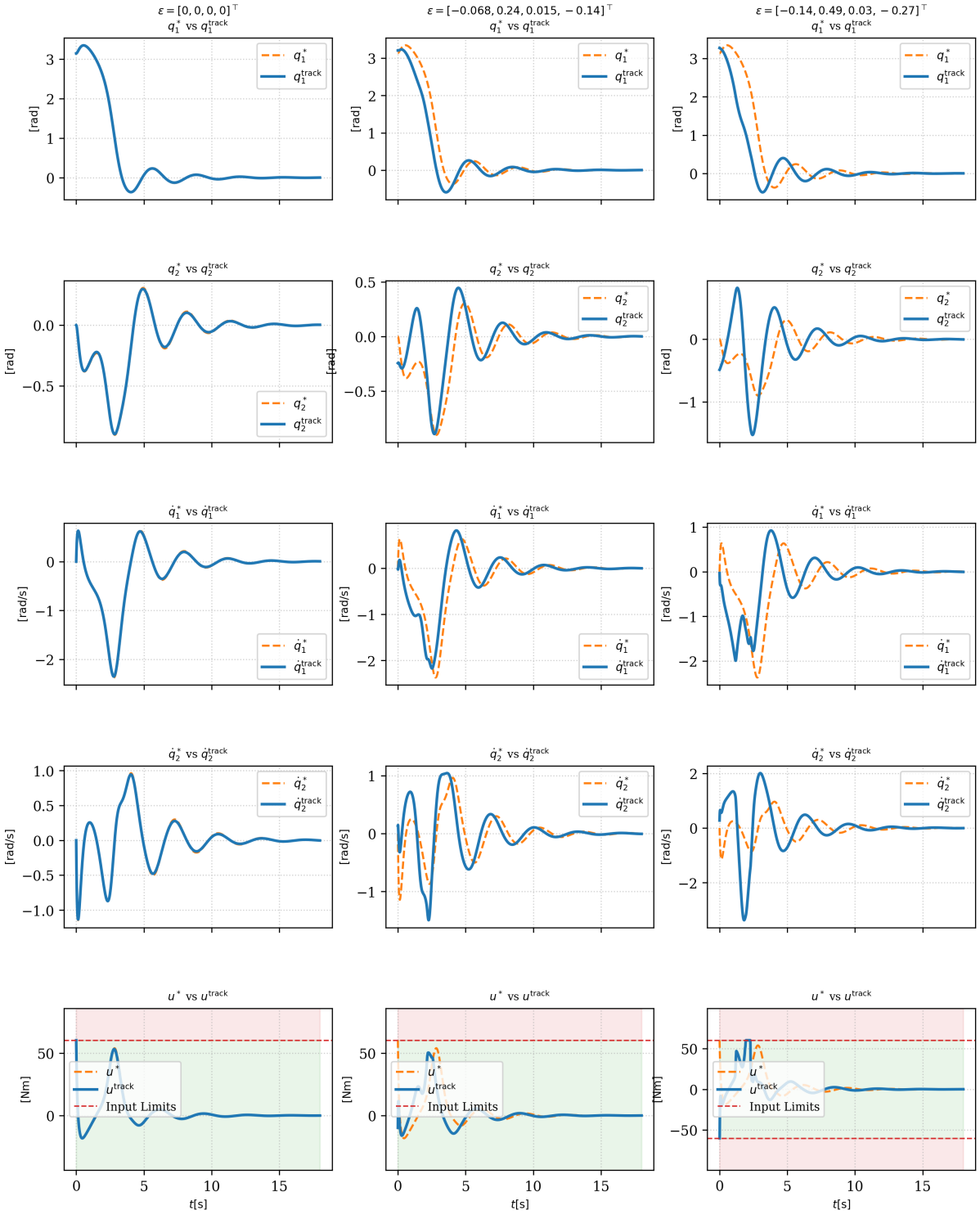


Figure 20: Trajectory tracking on a dive maneuver

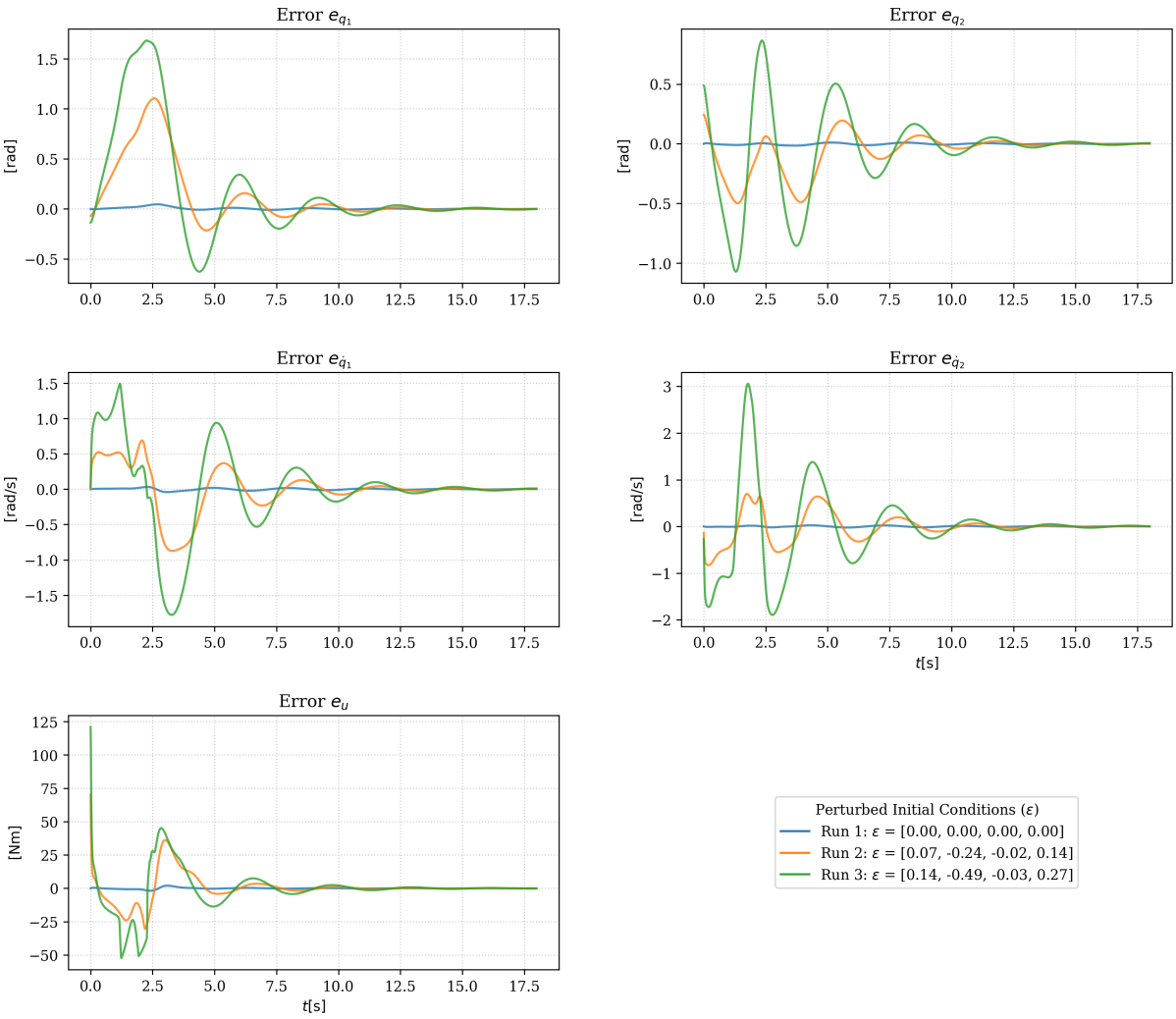


Figure 21: Error analysis during MPC tracking of dive maneuver

Trajectory Comparison: MPC Trajectory Tracking

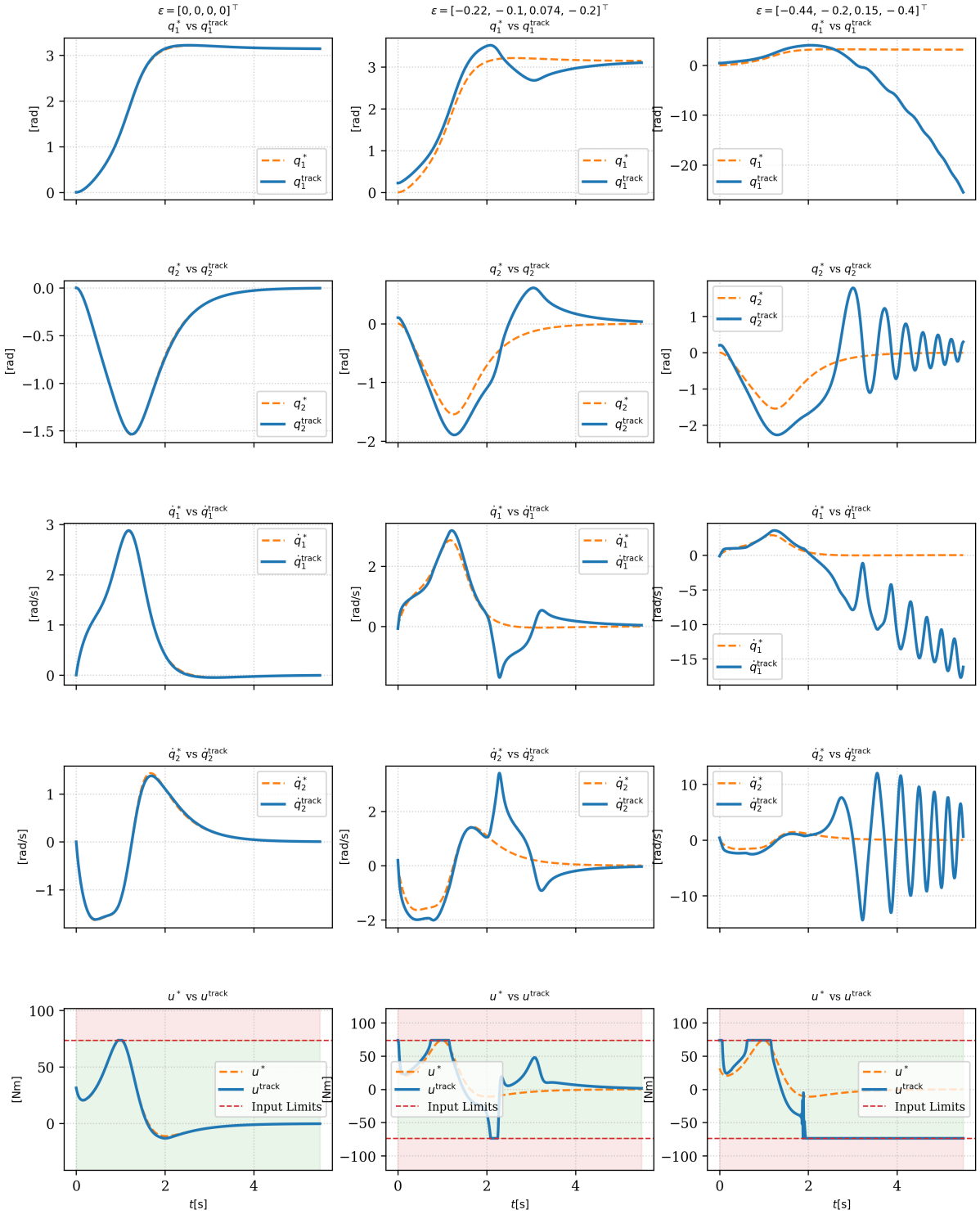


Figure 22: Trajectory tracking on a swing up maneuver

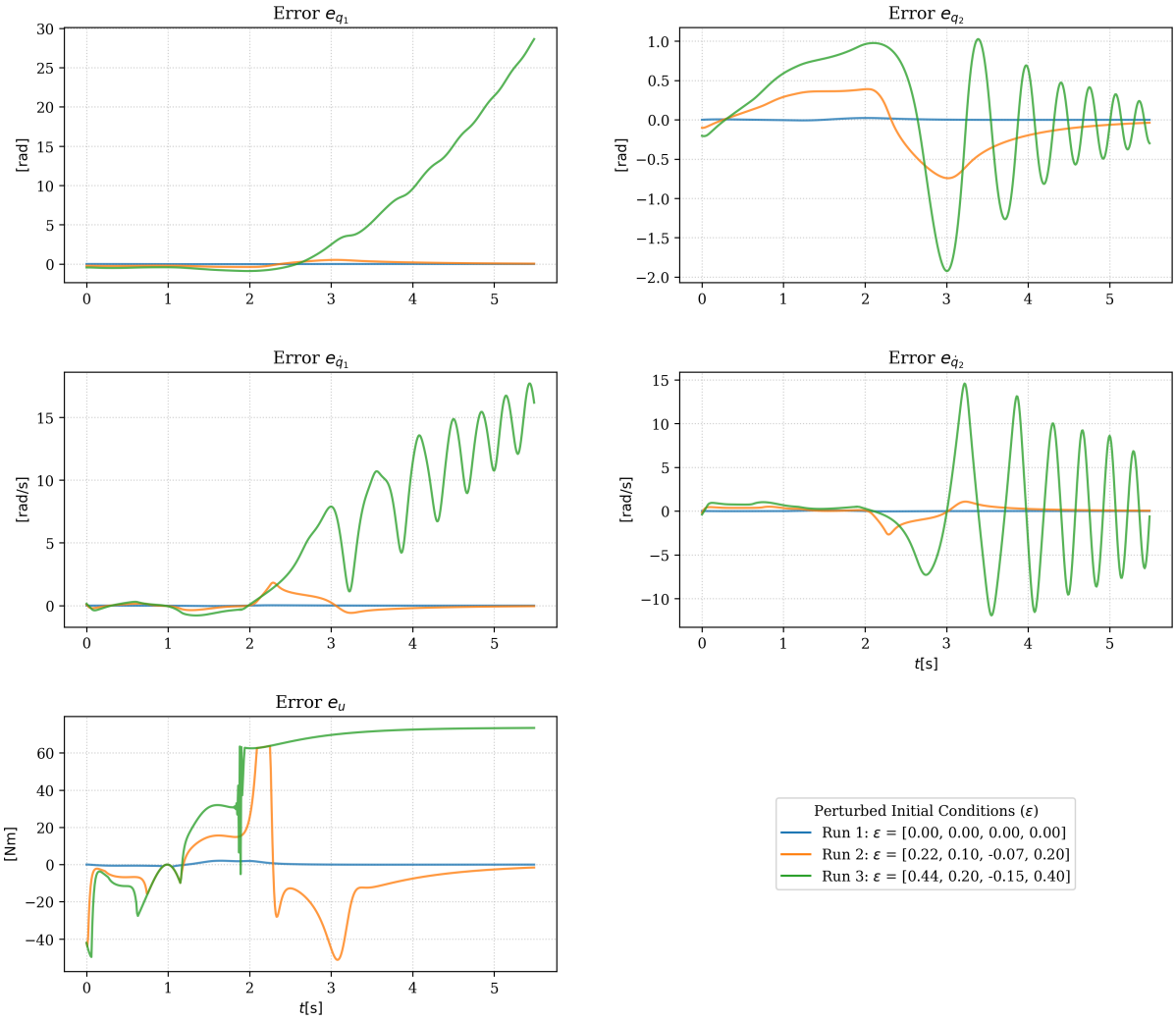


Figure 23: Error analysis during MPC tracking of swing up maneuver

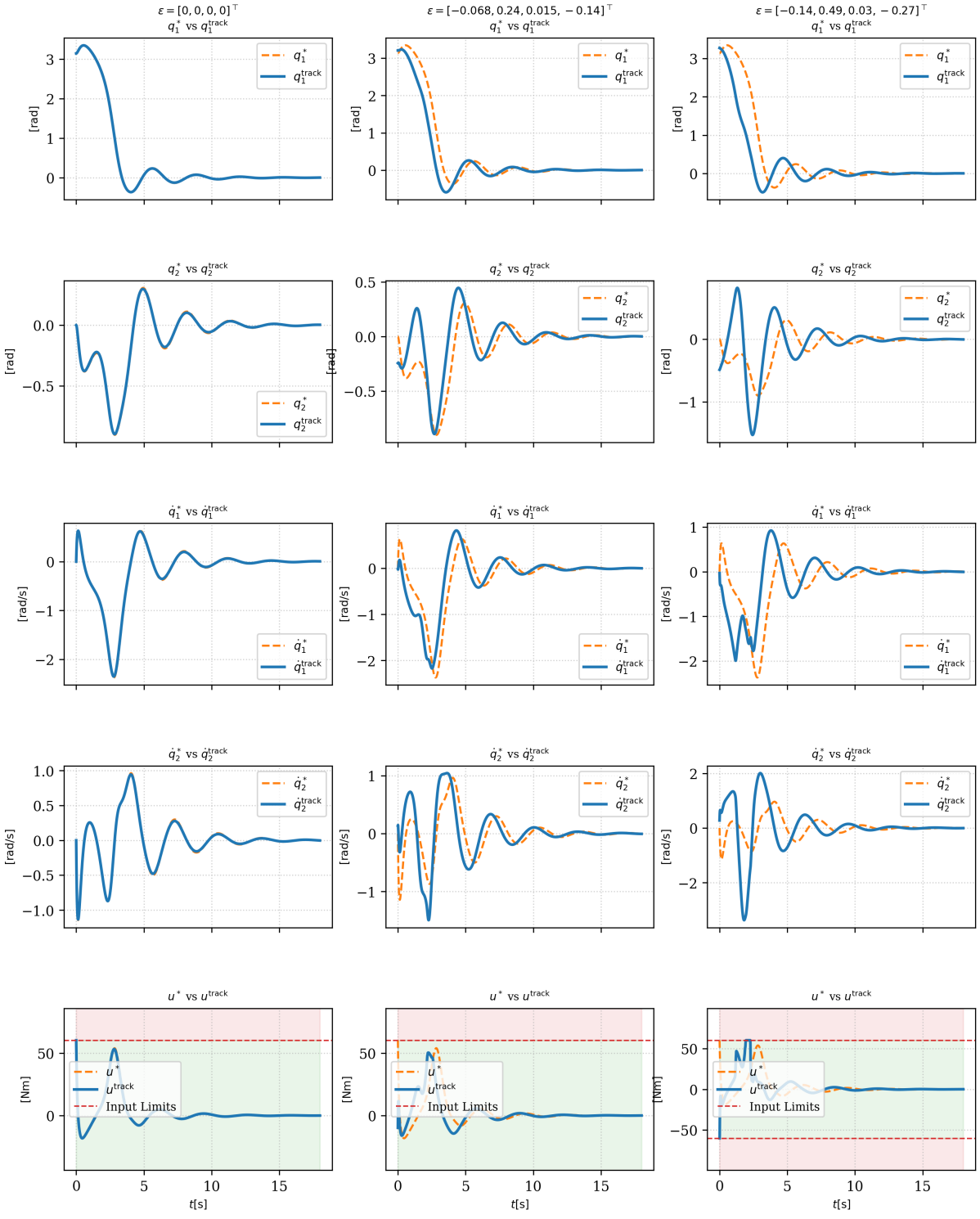


Figure 24: Trajectory tracking on a elbow up maneuver

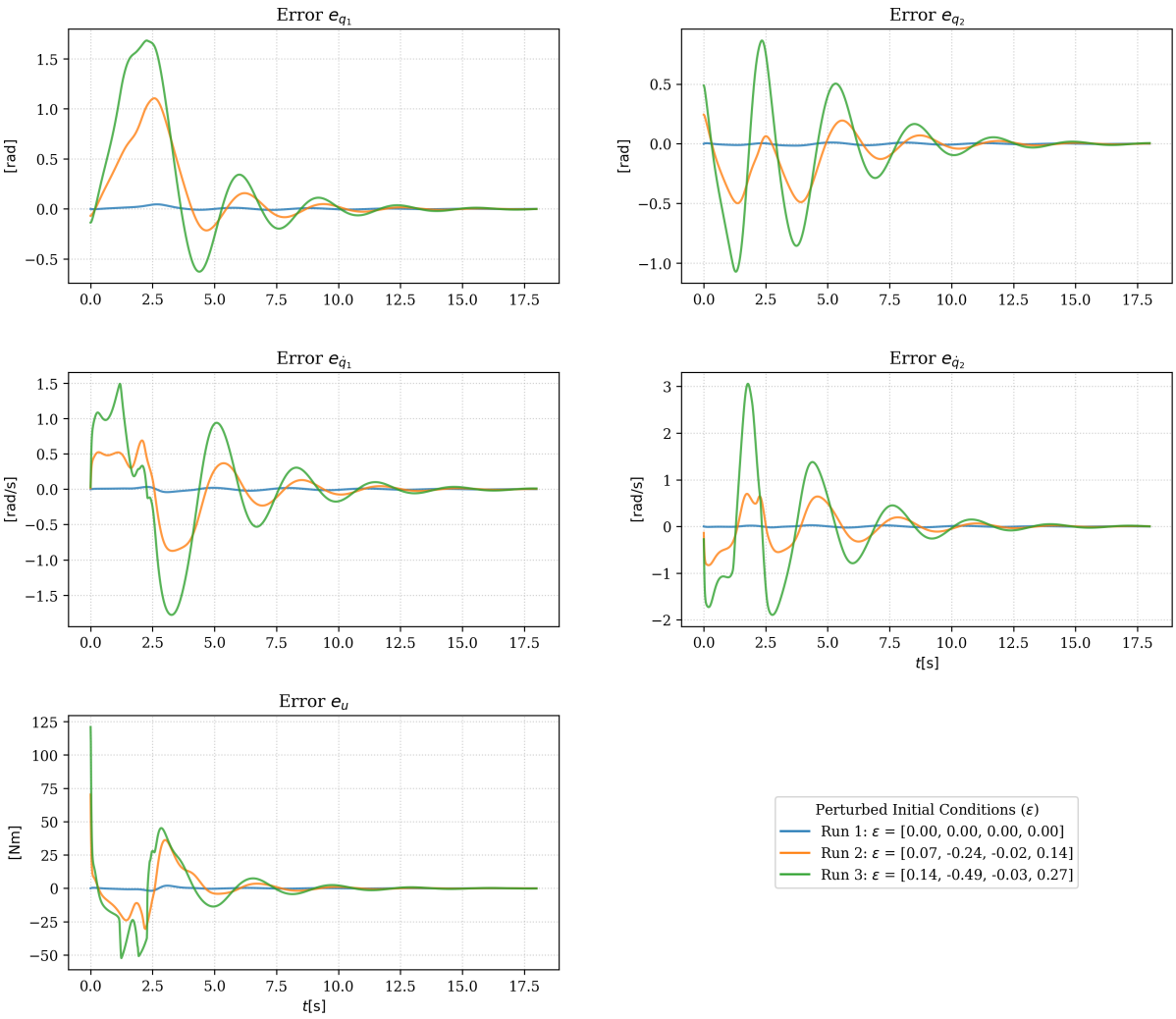


Figure 25: Error analysis during MPC tracking of a elbow maneuver

6 Task 5: Animation

6.1 Visualization

An animation suite was developed (‘animation.py‘) to visualize the results. It displays the double pendulum link, the reference trajectory (ghost), and the actual tracked path. This visual verification was crucial for debugging the "elbow-flip" behaviors inherent in the nonlinear stiffness model.

7 Conclusion

This project successfully demonstrated the control of a flexible under-actuated link.

- The **analytic Jacobian** and **RK4 discretization** provided the necessary precision for optimization.
- The **DARE-based terminal cost** significantly improved the stability of finite-horizon approximations.
- The **Dynamic Horizon Extension** strategy was the key enabler for generating complex trajectories, effectively bypassing local minima issues common in nonlinear optimization.
- Finally, **LQR and MPC** proved effective for tracking, with MPC offering the critical advantage of handling actuator constraints.

x

8 Appendix

8.1 Models matrices and parameters

$$M(q) = \begin{bmatrix} I_1 + I_2 + l_{c1}^2 m_1 + m_2(l_1^2 + 2l_1 l_{c2}\cos(\theta_2) + l_{c2}^2) & I_2 + l_{c2} m_2 (l_1 \cos(\theta_2) + l_{c2}) \\ I_2 + l_{c2} m_2 (l_1 \cos(\theta_2) + l_{c2}) & I_2 + l_{c2}^2 m_2 \end{bmatrix}$$

$$C(q, \dot{q}) = \begin{bmatrix} -l_1 l_{c2} m_2 \dot{\theta}_2 \sin(\theta_2) & -l_1 l_{c2} m_2 (\dot{\theta}_1 + \dot{\theta}_2) \sin(\theta_2) \\ l_1 l_{c2} m_2 \dot{\theta}_1 \sin(\theta_2) & 0 \end{bmatrix}$$

$$G(q) = \begin{bmatrix} g l_{c1} m_1 \sin(\theta_1) + g m_2 (l_1 \sin(\theta_1) + l_{c2} \sin(\theta_1 + \theta_2)) \\ g m_2 l_{c2} \sin(\theta_1 + \theta_2) + 3k \sin(\theta_2) \cos^2(\theta_2) \end{bmatrix}$$

$$F = \begin{bmatrix} f_1 & 0 \\ 0 & f_2 \end{bmatrix}$$

Symbol	Description	Value	Unit
m_1, m_2	Mass of link 1 and 2	2	kg
l_1, l_2	Length of link 1 and 2	1.5	m
r_1, r_2	Distance to center of mass	0.75	m
I_1, I_2	Moment of inertia	1.5	$\mathrm{kg\,m^2}$
g	Gravitational acceleration	9.81	$\mathrm{m/s^2}$
f_1, f_2	Viscous friction coefficient	1.0	$\mathrm{N\,m\,s/rad}$
k	Nonlinear angular stiffness	0.6	$\mathrm{N\,m/rad}$

Table 1: Model Parameters (Set 2)

8.2 Analytical Derivation of the Jacobian

This appendix details the analytical computation of the Jacobian matrices A and B used in the linearization of the flexible double pendulum dynamics.

8.2.1 State-Space Formulation

Consider the state space formulation given in Section 1.

$$\dot{x} = \begin{bmatrix} \dot{q} \\ \ddot{q} \end{bmatrix} = \begin{bmatrix} \dot{q} \\ M(q)^{-1} \left(\tau - C(q, \dot{q})\dot{q} - G(q) - F\dot{q}\right) \end{bmatrix} \tag{12}$$

$$\dot{x} = f(x, u)$$

To linearize the system about a point $(\bar{x}, \bar{u})$, we compute the State and Input Jacobians:

$$A = \left.\frac{\partial f(x,u)}{\partial x}\right|_{\bar{x},\bar{u}} \in \mathbb{R}^{4\times 4} \quad B = \left.\frac{\partial f(x,u)}{\partial u}\right|_{\bar{x},\bar{u}} \in \mathbb{R}^{4\times 1}$$

The state Jacobian $A \in \mathbb{R}^{4\times 4}$ has the following block structure:

$$A = \begin{bmatrix} \frac{\partial \dot{q}}{\partial q} & \frac{\partial \dot{q}}{\partial \dot{q}} \\ \frac{\partial \ddot{q}}{\partial q} & \frac{\partial \ddot{q}}{\partial \dot{q}} \end{bmatrix} = \begin{bmatrix} 0_{2\times 2} & I_{2\times 2} \\ A_{21} & A_{22} \end{bmatrix} \tag{13}$$

8.2.2 Block A_{22}: Partial Derivative w.r.t. Velocities

Let $\eta = \tau - C(q,\dot{q})\dot{q} - G(q) - F\dot{q}$ be the net torque. Then $\ddot{q} = M^{-1}\eta$. Since M and G depend only on positions q, the derivative with respect to $\dot{q}$ is:

$$A_{22} = \frac{\partial \ddot{q}}{\partial \dot{q}} = M^{-1}\left(-\frac{\partial (C(q,\dot{q})\dot{q})}{\partial \dot{q}} - F\right) \tag{14}$$

8.2.3 Block A_{21}: Partial Derivative w.r.t. Positions

The derivative with respect to q involves the chain rule on both the inverse mass matrix and the force vector η:

$$A_{21} = \frac{\partial \ddot{q}}{\partial q} = \underbrace{\frac{\partial (M^{-1})}{\partial q}\eta}_{\text{term 1}} + \underbrace{M^{-1}\frac{\partial \eta}{\partial q}}_{\text{term 2}} \tag{15}$$

Term 1: Derivative of Inverse Mass Matrix Using the identity $\frac{\partial M^{-1}}{\partial \alpha} = -M^{-1}\frac{\partial M}{\partial \alpha}M^{-1}$, the first term is computed column-wise for q_1 and q_2:

$$\frac{\partial (M^{-1})}{\partial q_i}\eta = -M^{-1}\frac{\partial M}{\partial q_i}M^{-1}\eta = -M^{-1}\frac{\partial M}{\partial q_i}\ddot{q} \tag{16}$$

Term 2: Derivative of Forces The derivative of the net torque η with respect to q is:

$$\frac{\partial \eta}{\partial q} = -\frac{\partial (C(q,\dot{q})\dot{q})}{\partial q} - \frac{\partial G}{\partial q} \tag{17}$$

8.2.4 Jacobian Matrix B

The input Jacobian $B \in \mathbb{R}^{4\times 1}$ is straightforward, as only the torque τ depends on u:

$$B = \begin{bmatrix} \frac{\partial \dot{q}}{\partial u} \\ \frac{\partial \ddot{q}}{\partial u} \end{bmatrix} = \begin{bmatrix} 0_{2\times 1} \\ M^{-1}\frac{\partial \tau}{\partial u} \end{bmatrix} \tag{18}$$

Since $\tau = [u, 0]^\top$, $\frac{\partial \tau}{\partial u} = [1, 0]^\top$. Thus, the bottom block of B is simply the first column of M^{-1}.

8.2.5 Explicit Derivative Matrices

For completeness, the non-zero partial derivatives implemented in the code are listed below. Let $s_i = \sin(q_i)$, $c_i = \cos(q_i)$, $s_{ij} = \sin(q_i + q_j)$, $c_{ij} = \cos q_i + q_j$.

Gravity Gradient

$$\frac{\partial G}{\partial q} = \begin{bmatrix} g(m_1 l_{c1} + m_2 l_1)c_1 + m_2 g l_{c2} c_{12} & m_2 g l_{c2} c_{12} \\ m_2 g l_{c2} c_{12} & m_2 g l_{c2} c_{12} + 3k(c_2^2 - 2s_2^2)c_2 \end{bmatrix} \tag{19}$$

Coriolis Velocity Gradient

$$\frac{\partial (C\dot{q})}{\partial \dot{q}} = 2m_2 l_1 l_{c2} s_2 \begin{bmatrix} -\dot{q}_2 & -(\dot{q}_1 + \dot{q}_2) \\ \dot{q}_1 & 0 \end{bmatrix} \tag{20}$$

Mass Matrix Gradient

$$\frac{\partial M}{\partial q_2} = -m_2 l_1 l_{c2} s_2 \begin{bmatrix} 2 & 1 \\ 1 & 0 \end{bmatrix} \tag{21}$$

8.3 Derivation of Discrete-Time Jacobians

This appendix details the linearization of the discrete-time dynamics obtained via numerical integration. Let the continuous-time dynamics be $\dot{x} = f(x, u)$, and let the discrete-time system be denoted by:

$$x_{k+1} = F(x_k, u_k) \tag{22}$$

where F is the numerical integration map over a time step Δt. We seek the discrete-time state Jacobian A_d and input Jacobian B_d:

$$A_d = \left.\frac{\partial F}{\partial x}\right|_{x_k,u_k}, \quad B_d = \left.\frac{\partial F}{\partial u}\right|_{x_k,u_k} \tag{23}$$

Let the continuous-time Jacobians evaluated at a specific state x and input u be denoted as $A_c(x, u) = \nabla_x f(x, u)^\top$ and $B_c(x, u) = \nabla_u f(x, u)^\top$.

8.3.1 Forward Euler Discretization

The Forward Euler method approximates the dynamics as:

$$x_{k+1} = x_k + \Delta t f(x_k, u_k) \tag{24}$$

Differentiating with respect to x_k and u_k yields the standard first-order approximation:

$$A_d = I + \Delta t A_c(x_k, u_k) \tag{25}$$
$$B_d = \Delta t B_c(x_k, u_k) \tag{26}$$

8.3.2 Runge-Kutta 4 (RK4) Discretization

The explicit RK4 integration scheme is defined by:

$$x_{k+1} = x_k + \frac{\Delta t}{6}(k_1 + 2k_2 + 2k_3 + k_4) \tag{27}$$

where the intermediate slopes k_i are:

$$\begin{aligned} k_1 &= f(x_k, u_k) \\ k_2 &= f(x_k + 0.5\Delta t k_1, u_k) \\ k_3 &= f(x_k + 0.5\Delta t k_2, u_k) \\ k_4 &= f(x_k + \Delta t k_3, u_k) \end{aligned}$$

To find A_d and B_d, we apply the chain rule recursively to the slopes k_i.

State Jacobian A_d Differentiating the update equation with respect to x_k:

$$A_d = I + \frac{\Delta t}{6}\left(\frac{\partial k_1}{\partial x} + 2\frac{\partial k_2}{\partial x} + 2\frac{\partial k_3}{\partial x} + \frac{\partial k_4}{\partial x}\right) \tag{28}$$

Let $x^{(i)}$ denote the argument of f for the calculation of k_i. The gradients are derived recursively:

$$\frac{\partial k_1}{\partial x} = A_c(x_k, u_k) \tag{29}$$
$$\frac{\partial k_2}{\partial x} = A_c(x^{(2)}, u_k)\left(I + 0.5\Delta t\frac{\partial k_1}{\partial x}\right) \tag{30}$$
$$\frac{\partial k_3}{\partial x} = A_c(x^{(3)}, u_k)\left(I + 0.5\Delta t\frac{\partial k_2}{\partial x}\right) \tag{31}$$
$$\frac{\partial k_4}{\partial x} = A_c(x^{(4)}, u_k)\left(I + \Delta t\frac{\partial k_3}{\partial x}\right) \tag{32}$$

Input Jacobian B_d Differentiating the update equation with respect to u_k:

$$B_d = \frac{\Delta t}{6}\left(\frac{\partial k_1}{\partial u} + 2\frac{\partial k_2}{\partial u} + 2\frac{\partial k_3}{\partial u} + \frac{\partial k_4}{\partial u}\right) \tag{33}$$

Note that u_k appears both directly in f and indirectly through the state perturbation in the intermediate steps. The recursive gradients are:

$$\frac{\partial k_1}{\partial u} = B_c(x_k, u_k) \tag{34}$$

$$\frac{\partial k_2}{\partial u} = A_c(x^{(2)}, u_k)\left(0.5\Delta t\frac{\partial k_1}{\partial u}\right) + B_c(x^{(2)}, u_k) \tag{35}$$

$$\frac{\partial k_3}{\partial u} = A_c(x^{(3)}, u_k)\left(0.5\Delta t\frac{\partial k_2}{\partial u}\right) + B_c(x^{(3)}, u_k) \tag{36}$$

$$\frac{\partial k_4}{\partial u} = A_c(x^{(4)}, u_k)\left(\Delta t\frac{\partial k_3}{\partial u}\right) + B_c(x^{(4)}, u_k) \tag{37}$$

This exact linearization of the RK4 integrator ensures that the discrete-time Jacobians accurately reflect the curvature of the numerical integration path, which is critical for the stability of Newton-based optimal control solvers.

8.4 Newton Method with Dynamic Horizon Extension algorithm

Algorithm 1 Newton Method with Dynamic Horizon Extension

Require: Dynamics $x_{t+1} = f(x_t, u_t)$, Start x_{start}, Goal (x_{eq}, u_{eq}), Costs Q, R, Initial Horizon T, Max Horizon T_{max}, Extension ΔT, Tolerances $\epsilon_{desc}, \epsilon_{term}$.
Ensure: Optimal Trajectory $\mathbf{x}^*, \mathbf{u}^*$

Initialization:
Compute P_N solving DARE(A_{eq}, B_{eq}, Q, R) at x_{eq}.
Generate Reference $\mathbf{x}^{ref}, \mathbf{u}^{ref}$ (e.g., Poly5).
Initialize guess $\mathbf{x}^{(0)}, \mathbf{u}^{(0)}$ (Warm start or Equilibrium).
$k \leftarrow 0$ ▷ Optimization Iteration
$j \leftarrow 0$ ▷ Extension Phase
while $T < T_{max}$ **do** ▷ **Outer Loop: Horizon Extension**
  repeat ▷ **Inner Loop: Newton Optimization**
    $J^{(k)} \leftarrow$ Compute Total Cost
    // Backward Pass
    $P_N \leftarrow P_f, \quad p_N \leftarrow \nabla_x \ell_f(x_N)$
    for $t = N-1$ **to** 0 **do**
      Linearize f to get A_t, B_t
      Compute gradients q_t, r_t and Hessian H_t of cost
      $Q_{xx}, Q_{uu}, Q_{xu} \leftarrow$ Expansion of Hamiltonian
      $K_t \leftarrow -Q_{uu}^{-1} Q_{ux}$ ▷ Feedback Gain
      $\sigma_t \leftarrow -Q_{uu}^{-1} Q_u$ ▷ Feedforward term
      Update P_t, p_t (Riccati equations)
    end for
    // Line Search (Armijo)
    Find α such that cost decreases sufficiently
    // Forward Pass
    $x_0 \leftarrow x_{start}$
    for $t = 0$ **to** $N-1$ **do**
      $u_t \leftarrow u_t^{(k)} + \alpha\sigma_t + K_t(x_t - x_t^{(k)})$
      $x_{t+1} \leftarrow f(x_t, u_t)$
    end for
    Update $\mathbf{x}^{(k+1)}, \mathbf{u}^{(k+1)}$, increment k
  until $\|\text{Descent}\| < \epsilon_{desc}$
  // Check Terminal Constraint
  if $\|x_N - x_{eq}\| < \epsilon_{term}$ **then**
    Break ▷ Solution found
  else
    Extend Horizon: $T \leftarrow T + \Delta T$
    Pad $\mathbf{x}^{ref}, \mathbf{u}^{ref}$ with x_{eq}, u_{eq}
    Pad current solution $\mathbf{x}, \mathbf{u}$ with simulation at u_{eq}
  end if
end while
return $\mathbf{x}, \mathbf{u}$